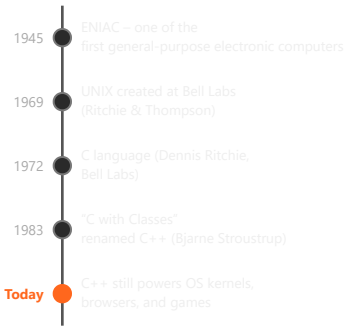
MODULE 1

PROGRAMMING & C++



MODULE 1

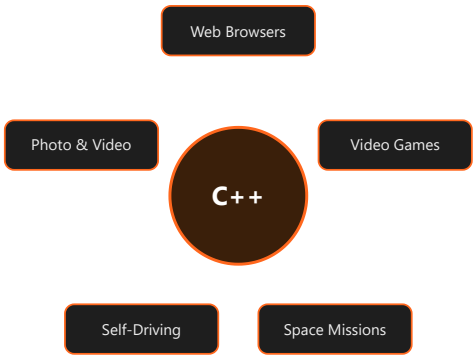
COURSE OVERVIEW

1. **History of C/C++**
2. **From Code to Machine**
3. **History of C++**
4. **I/O & Variables**



Official ISO C++ logo (Source: **Wikipedia, ISO C++ logo**)

WHERE YOU ALREADY USE C++



C++ BEHIND EVERYDAY APPS

Chromium	Powers the Chrome browser	Web Browsers
Unreal Engine	Built Fortnite & more	Video Games
Ingenuity	NASA's Mars helicopter	Space Missions

Sources: **Wikipedia, Chromium**, **Epic Games, Unreal Engine**, **NASA/JPL, Ingenuity**







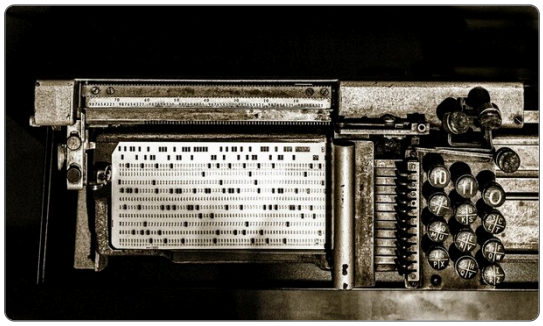






BEFORE OPERATING SYSTEMS

- **1940s–1950s**: switches, punch cards, raw code
- One program controlled the whole machine
- Devices, stop/start handled manually
- Very slow, error-prone



BELL LABS AND AT&T

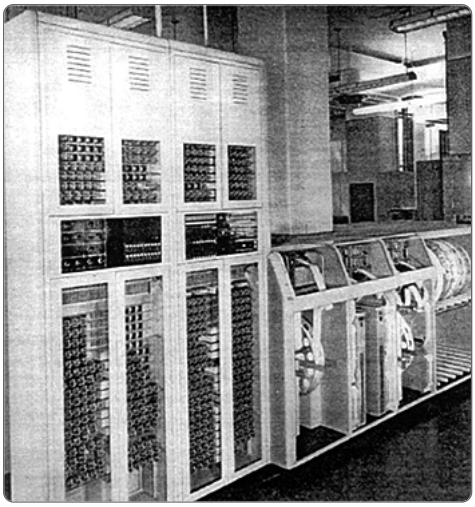
- Bell Labs was the research arm of **AT&T** (**founded 1925**).
- In the **late 1960s**, they needed an **operating system** to **manage** computers for **communication networks** (manage = schedule programs, allocate CPU & memory, control devices, share files).
- **1969–1971**: Led to the creation of **UNIX** as a flexible system for research and telecom needs.



HISTORY OF C/C++

- Early 1970s at Bell Labs (Dennis Ritchie)
- System programming for UNIX



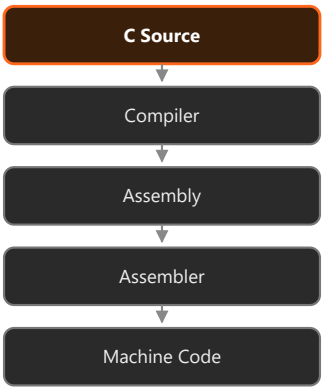


WHY C?

- Close to hardware, portable
- Small runtime, fast compilation

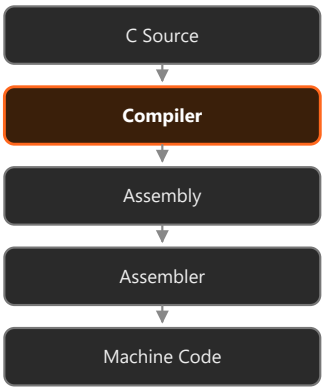
QUESTIONS?

FROM CODE TO MACHINE



C CODE

```
int totalCredits(int a, int b) { return a + b
```

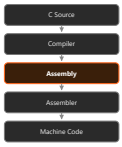


COMPILER

```
$ gcc -S totalCredits.c -o totalCredits.s
```

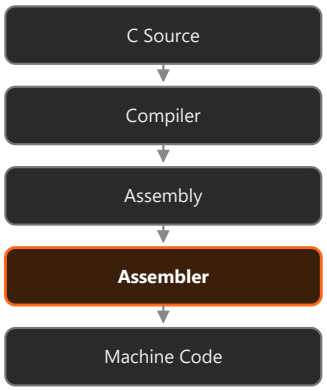
Translates C source into assembly

ASSEMBLY




```
totalCredits: mov eax, edi ; a into eax add eax, esi ; add b to eax ret ; return
```

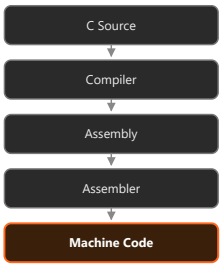
x86-64 System V calling convention



ASSEMBLER

```
$ as totalCredits.s -o totalCredits.o
```

Assembles into machine code (object file)



MACHINE CODE

```
55 ; push rbp 89 f8 ; mov eax, edi 01 f0 ; add eax, esi 5d ; pop rbp c3
```

Compiled with gcc -O1 on x86-64

ENROLL STUDENT (C CODE)

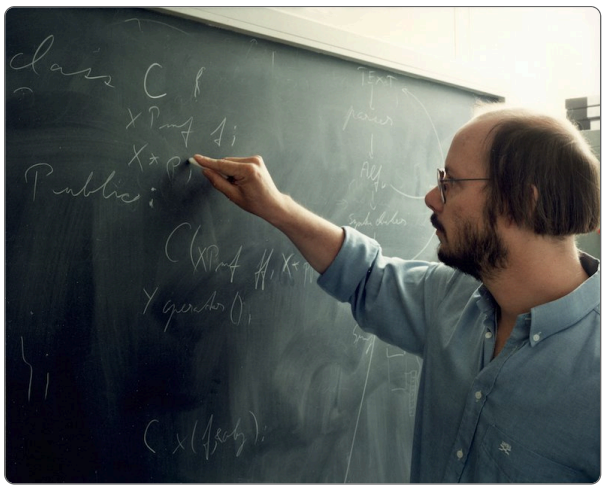
```
#include <stdio.h> // Enroll a student void enroll_student() { printf("Linking...\n"); printf("E
```

QUESTIONS?

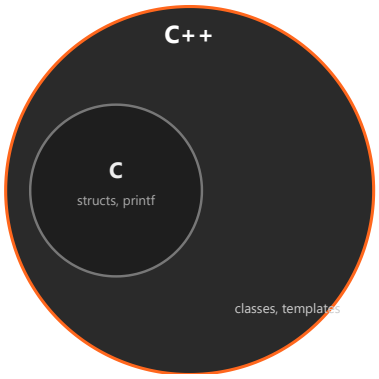
HISTORY OF C++

HISTORY OF C++

- 1980s: Bjarne Stroustrup
- “C with Classes” → C++



C IS A SUBSET OF C++



Every valid C program is (almost) valid C++



WHY C++?

- Object-oriented programming
- Performance with abstraction

QUICK CHECK

Which is a subset of the other?

1. C++ is a subset of C
2. C is a subset of C++
3. They are unrelated

B. C is a subset of C++.

QUESTIONS?

I/O & VARIABLES

BASIC I/O

```
#include <iostream> int main(){ int credits; std::cout << "Credits enrolled? "; std::cin >> credits; std::cout <
```

QUICK CHECK

Which operator reads user input?

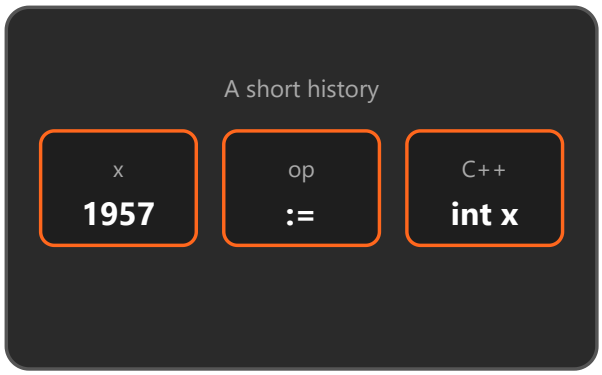
1. << sends output
2. >> reads input
3. == compares values

B. cin >> reads input.

VARIABLES

HISTORY OF THE VARIABLE

- 1957: FORTRAN adds named data
- 1958: ALGOL adds := for it
- Every language has kept it



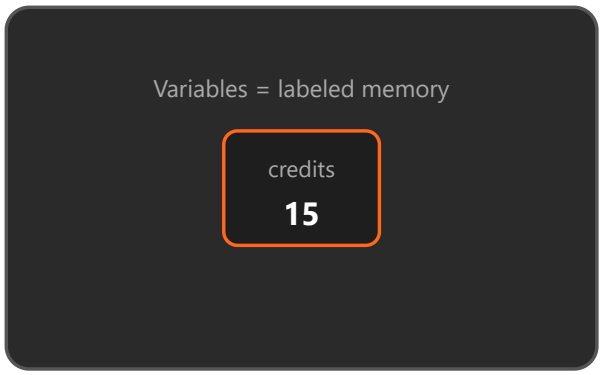
WHY VARIABLES?

- Store a value, reuse it
- Give data a name

```
// No var, repeat cout << 15; cout << 15; // Reuse the
```

WHAT IS A VARIABLE?

- A named spot in memory
- Holds one value at a time
- The value can change



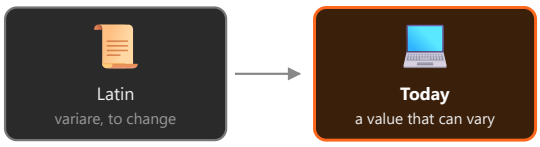
ANATOMY OF A VARIABLE

- Type: what kind of value
- Name: how you refer to it
- Value: what's stored now

```
int credits = 15; // ^ ^ ^ // type name va
```

WORD ORIGIN: "VARIABLE"

- Latin: variare, to change
- Math: an unknown quantity
- 1957: FORTRAN's named data



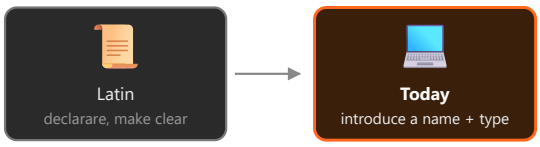
DECLARING A VARIABLE

- Tells the compiler: this name
- exists, and holds this type
- No value yet

```
int credits; // no value ye
```

WORD ORIGIN: "DECLARE"

- Latin: declarare, make clear
- You're telling the compiler
- "this name will exist"



CODE IT: DECLARE A VARIABLE

- Declare an int named year
- No value yet
- How would you write it?

```
int year;
```

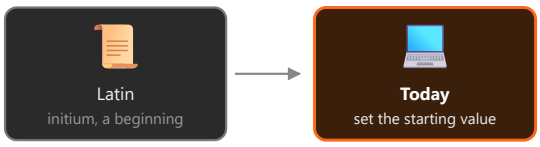
INITIALIZING A VARIABLE

- Declare AND give a value
- Happens in one statement
- Best practice: always do this

```
int credits = 15; // declared & s
```

WORD ORIGIN: "INITIALIZE"

- Latin: initium, a beginning
- "Set the starting value"
- Happens at declaration



QUICK CHECK

What's the difference: declare vs. initialize?

1. No difference at all
2. Initialize also sets a value
3. Declare also sets a value

B. Initializing = declaring + giving it a starting value.

CODE IT: INITIALIZE A VARIABLE

- Initialize gpa to 3.5
- Type: double
- How would you write it?

```
double gpa = 3.5;
```

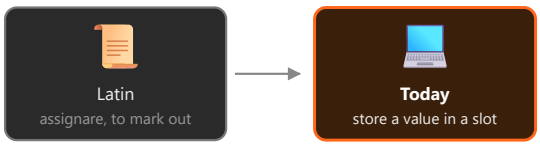
ASSIGNING A VALUE

- = is the assignment operator
- Stores a value in a variable
- Variable must exist first

```
int credits; credits = 15; // after decl
```

WORD ORIGIN: "ASSIGN"

- Latin: assignare, mark out
- 1958: ALGOL's := operator
- C++ shortens it to just =



CODE IT: ASSIGN A VALUE

- Declare age (int) first
- Then assign it 20
- How would you write it?

```
int age; age = 20;
```

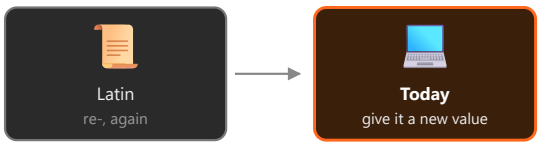
REASSIGNING A VARIABLE

- Change the stored value
- Old value is overwritten
- Same variable, new value

```
int credits = 15; credits = 18; // old value g
```

WORD ORIGIN: "REASSIGN"

- Latin: re-, again + assign
- "Assign a new value again"
- Same variable, new value



QUICK CHECK

What happens when you reassign a variable?

1. A new variable is created
2. The old value is overwritten
3. Nothing, it's an error

B. Reassigning overwrites the old value in place.

CODE IT: REASSIGN A VARIABLE

- credits starts at 12
- Reassign it to 16
- How would you write it?

```
int credits = 12; credits = 1
```

DECLARE, INITIALIZE, ASSIGN, REASSIGN

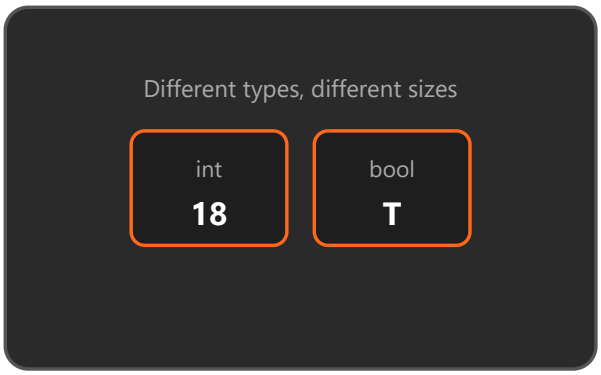
- int x; is declaring
- int x = 5; is initializing
- x = 9; is (re)assigning

```
int x; // declare x = 5; // assign int y = 5; // init y
```

TYPES OF VARIABLES

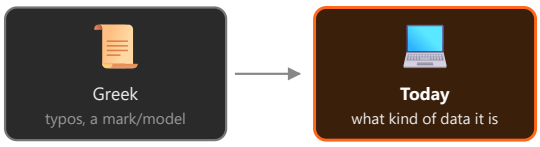
WHY DO TYPES MATTER?

- Type = what kind of data
- Sets the size in memory
- Sets what operations work



WORD ORIGIN: "TYPE"

- Greek: typos, a mark/model
- Defines a value's shape
- int, double, char, bool...



OPERATORS:
+ AND -

- + adds two values
- - subtracts two values
- Work like in math class

```
int a = 10, b = 3; int sum=a+b; // 13 int diff=a-b;
```

OPERATORS:
* AND /

- * multiplies two values
- / divides two values
- Division can drop decimals

```
int a = 10, b = 3; int prod=a*b; // 30 int quot=a/b;
```

TYPE: INT

- Whole numbers only
- No decimal point
- e.g., credits, a count

```
int credits = 15; int year = 202
```

INT IN PRACTICE

- Most common numeric type
- No fractions allowed
- 9/2 gives 4, not 4.5

```
int a = 9; int b = 2; int result = a / b; cout << result;
```

QUICK CHECK

Which university variable fits int best?

1. unitsCompleted
2. cumulativeGPA
3. studentEmail

A. unitsCompleted is a whole number, no decimals.

CODE IT: UNITS COMPLETED

- Declare units
- Set it to 45
- How would you write it?

```
int units=45;
```

TYPE: DOUBLE

- Numbers with decimals
- "Double precision" float
- e.g., gpa, price, average

```
double gpa = 3.8; double price = 19.9
```

DOUBLE IN PRACTICE

- Keeps the decimal part
- Use for any fraction
- 9.0 / 2 gives 4.5

```
double a = 9.0; double b = 2.0; double r = a / b; cout <<
```

QUICK CHECK

Which university variable fits double best?

1. tuitionBalance
2. semesterCount
3. isFullTime

A. tuitionBalance needs cents, a decimal value.

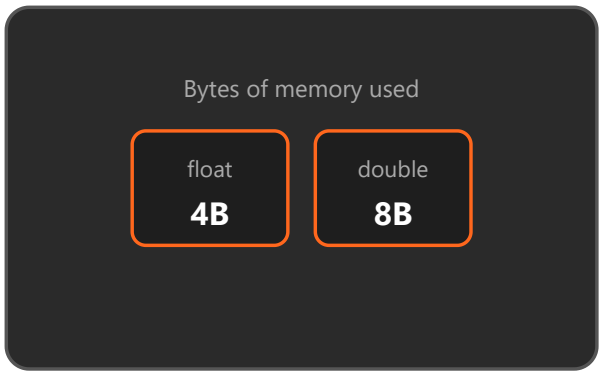
CODE IT: TUITION BALANCE

- Declare balance
- Set it to 450.75
- How would you write it?

```
double bal=450.75;
```

FLOAT VS. DOUBLE

- float: less precision
- double: more precision
- This course uses double



TYPE: CHAR

- One single character
- Wrapped in single quotes
- e.g., a grade letter

```
char grade = 'A'; char initial = 'S
```

QUICK CHECK

Which is a valid char value?

1. 'A'
2. "A"
3. A

A. char uses single quotes, one character.

QUICK CHECK

Which university variable fits char best?

1. letterGrade
2. studentID
3. creditHours

A. letterGrade is one character, like 'A'.

CODE IT: LETTER GRADE

- Declare letterGrade
- Set it to 'B'
- How would you write it?

```
char letterGrade='B';
```

TYPE: BOOL

- Only true or false
- Named after George Boole
- e.g., isEnrolled, passed

```
bool isEnrolled=true; bool passed = fal
```

BOOL IN PRACTICE

- Stores yes/no answers
- Used a lot in decisions
- if (isEnrolled) later on

```
bool ok=grade>=60; cout << ok; //
```

QUICK CHECK

Which university variable fits bool best?

1. isEnrolled
2. major
3. unitsTaken

A. isEnrolled is only ever true or false.

CODE IT: IS ENROLLED

- Declare isEnrolled
- Set it to true
- How would you write it?

```
bool isEnrolled=true;
```

TYPE: STRING

- Text: words, sentences
- Wrapped in double quotes
- Needs #include <string>

```
#include <string> string name="Alice
```

WHAT IS +=?

- Shorthand for reassignment
- x += 5 means x = x + 5
- Also -=, *=, /=, %=

```
int x = 10; x += 5; // x = x + 5 // x is now
```

COMPOUND ASSIGNMENT OPS

- += add and assign
- -= subtract and assign
- *=, /=, %= work the same

```
int credits = 12; credits += 3; // 15 credits -= 2; /
```

QUICK CHECK

What does x += 5; do?

1. Sets x to 5
2. Sets x to x + 5
3. Compares x to 5

B. x += 5; reassigns x to x + 5.

CODE IT:
RUNNING TOTAL

- Declare total = 0 (int)
- Add 10 using +=
- Print total
- How would you write it?

```
int total = 0; total += 10; cout << tot
```

CODE IT: COUNTDOWN SEATS

- Declare seats = 5 (int)
- Subtract 1 using -=
- Print seats
- How would you write it?

```
int seats = 5; seats -= 1; cout << sea
```

STRING IN PRACTICE

- Store any length of text
- Can combine with +
- e.g., building a message

```
string first = "Ana"; string last = "Lee"; string full =
```

QUICK CHECK

Which university variable fits string best?

1. studentName
2. birthYear
3. isGraduate

A. studentName holds text, a person's name.

CODE IT: STUDENT NAME

- Declare studentName
- Set it to "Maria"
- How would you write it?

```
string studentName = "Maria
```

QUICK CHECK

Which type would you use for a course name?

1. int
2. bool
3. string

C. string stores text like a course name.

NAMING RULES

- Start with a letter or _
- No spaces, no dashes
- Case-sensitive: age ≠ Age

```
int age; // valid int _temp; // valid int 2fast; // inv
```

NAMING CONVENTIONS

- Use descriptive names
- camelCase in C++
- Not just x, y, z

```
int studentCount; int sc;
```

CASE STYLES IN C++

- camelCase: variables
- PascalCase: class names
- ALL_CAPS: constants

```
int studentAge; class Student {}; const int MAX_A=
```

CHOOSING GOOD NAMES

- Be specific, not generic
- Avoid cryptic short names
- Loop counters (i, j) OK

```
int d; // unclear int daysLef
```

QUICK CHECK

Which naming style fits a class name?

1. camelCase
2. PascalCase
3. ALL_CAPS

B. PascalCase: Capitalize each word, e.g. Student.

QUICK CHECK

Which variable name is invalid in C++?

1. studentAge
2. _temp
3. 2ndTry

C. Names can't start with a digit.

CODE IT: A CLEARER NAME

- Rename a bad variable: d
- Make it clear and descriptive
- How would you write it?

```
int daysLeft;
```

WHY COMMENT CODE?

- Explains why, not just what
- Helps future you and others
- Ignored by the compiler

```
// A student's GPA double gpa = 3.
```

SINGLE-LINE COMMENTS

- Starts with //
- Runs to end of line
- Best for short notes

```
int credits = 15;
```

MULTI-LINE COMMENTS

- Starts /* ends */
- Spans several lines
- Good for longer notes

```
/* Explains the grade calc below
```

QUICK CHECK

Which starts a single-line comment?

1. //
2. /* */
3. #

A. // starts a comment to end of line.

CODE IT: ADD A COMMENT

- credits = 15
- Add a note: units
- How would you write it?

```
// units int credits = 15;
```

MULTIPLE VARIABLES, ONE LINE

- Same type, comma-separated
- Each still needs a value
- Saves a little typing

```
int hw1=10, hw2=8;
```

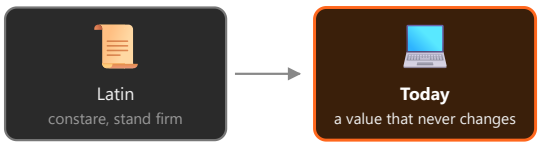
SWAPPING TWO VARIABLES

- a = 5, b = 9
- Need a 3rd, temp variable
- How would you swap them?

```
int a = 5, b = 9; int temp = a; a = b; b = temp; // now
```

WORD ORIGIN: "CONSTANT"

- Latin: constare, stand firm
- const means never reassigned
- Compiler enforces this



CONSTANTS: CONST

- A variable that can't change
- Set once, at initialization
- Protects a fixed value

```
const double PI=3.14; // PI = 3; // err
```

QUICK CHECK

What does the const keyword do?

1. Deletes the variable
2. Prevents reassignment
3. Makes it a string

B. const locks the value after initialization.

CODE IT: LOCK A CONSTANT

- Declare MAX, an int
- Lock it at 30, never changes
- How would you write it?

```
const int MAX = 30;
```

TRY IT YOURSELF

- Open onlinegdb.com
- Pick "C++" as the language
- Type, then click Run



TRY IT: DECLARE & PRINT

- Declare an int named age
- Assign it your age
- Print it with cout
- Run it at onlinegdb.com

```
int age; age = 20; cout << ag
```

TRY IT: INITIALIZE A DOUBLE

- Initialize a double gpa
- Give it a value like 3.5
- Print it with cout
- Run it at onlinegdb.com

```
double gpa = 3.5; cout << gp
```

TRY IT: REASSIGN A VARIABLE

- Initialize credits to 12
- Reassign it to 15
- Print it before and after
- Run it at onlinegdb.com

```
int credits = 12; cout << credits; credits = 15; cout
```

TRY IT: SWAP TWO VARIABLES

- Declare a = 3, b = 7
- Swap them with a temp var
- Print a and b after
- Run it at onlinegdb.com

```
int a = 3, b = 7; int temp = a; a = b; b = temp; cout
```

TRY IT: CHOOSE THE RIGHT TYPE

- Declare name (string)
- Declare gpa (double)
- Declare passed (bool)
- Run it at onlinegdb.com

```
string name = "Sam"; double gpa = 3.2; bool passed =
```

TRY IT: BUILD A MESSAGE

- Store first, last (string)
- Combine with +
- Print the full name
- Run it at onlinegdb.com

```
string first = "Sam"; string last = "Ray"; cout << fi
```

RESERVED WORDS

- Keywords the language owns
- int, class, return, etc.
- Can't be used as names

```
int int=5; //error! int total=5; //f
```

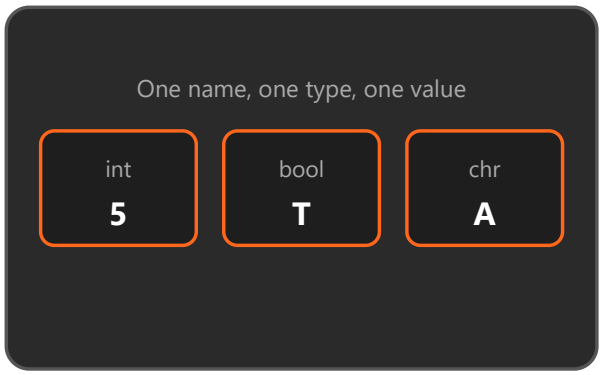
UNINITIALIZED VARIABLES

- Declared, but no value set
- Holds leftover "garbage"
- Always initialize first

```
int credits; cout << credits; // unpredictab
```

TYPES: QUICK SUMMARY

- int: whole numbers
- double: decimals
- char, bool, string: rest



TRY IT: COMPUTE AN AVERAGE

- Declare x, y (int)
- Declare a (double)
- Print a = (x+y)/2.0
- Run it at onlinegdb.com

```
int x = 10, y = 8; double a=(x+y)/2.0; cout <<
```

QUICK CHECK

Which line only DECLARES a variable, no value?

1. int x = 5;
2. int x;
3. x = 5;

B. int x; declares it with no value yet.

EXPRESSIONS

```
int hw1 = 10, hw2 = 3; int sum = hw1 + hw2; int product = hw1 * hw2; double avg = (hw1 + hw2
```

QUICK CHECK

What does the % operator return?

1. A percentage
2. The remainder (modulo)
3. A random number

B. The remainder of division.

QUESTIONS?

WHERE WE ARE: MODULE 1

- Module 1 done: C++ basics
- Next: Variables & I/O